Regno: 26MAG0064
Name: ADITYA SOGANI

```
"""
Lab Assignment: Linear Regression using Stochastic Gradient Descent (SGD)
Dataset: CIC-IDS2017 - WebAttacks-Thursday + Botnet-Friday combined
Feature (x): Total Fwd Packets | Target (y): Fwd Packets Length Total
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
# ----------------------------
# 1) Load the dataset
# ----------------------------
webattacks = pd.read_csv("WebAttacks-Thursday-no-metadata.csv")
botnet = pd.read_csv("Botnet-Friday-no-metadata.csv")
df = pd.concat([webattacks, botnet], ignore_index=True)
print("Combined shape:", df.shape)
 """output:
 Combined shape: (10000, 78)
 """

# ----------------------------
# 2) Select feature (x) and target (y)
# ----------------------------
# Simple linear regression: one input feature predicting one continuous target.
# More forward packets in a flow should roughly mean more total forward bytes.
data = df[["Total Fwd Packets", "Fwd Packets Length Total"]].copy()
data = data.replace([np.inf, -np.inf], np.nan).dropna()
# Remove extreme outliers (top 1%) so a handful of massive flows don't dominate
# the loss and destabilize SGD -- standard cleanup step before regression.
q99_x = data["Total Fwd Packets"].quantile(0.99)
q99_y = data["Fwd Packets Length Total"].quantile(0.99)
data = data[(data["Total Fwd Packets"] <= q99_x) & (data["Fwd Packets Length Total"]
<= q99_y)]
# Subsample for a manageable, fast per-sample SGD loop
data = data.sample(n=5000, random_state=42).reset_index(drop=True)
print("Working sample shape (after outlier removal):", data.shape)

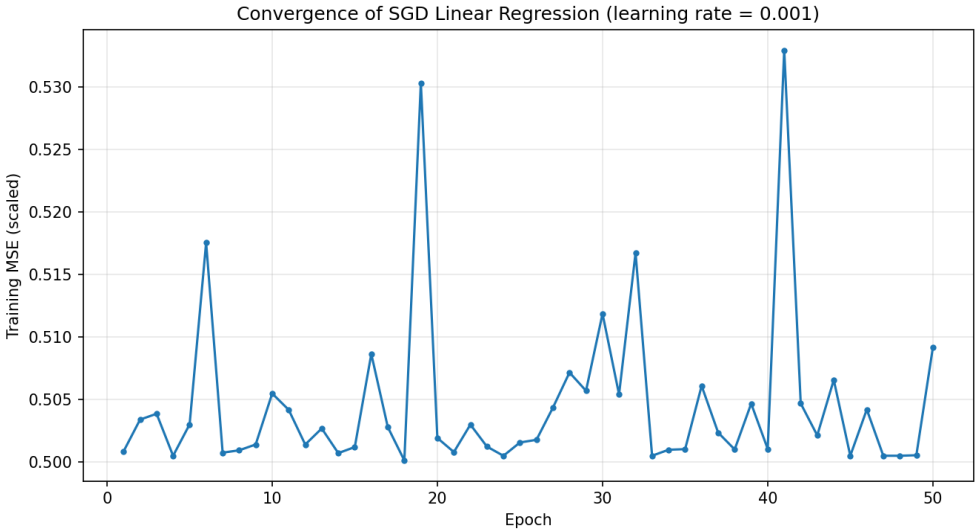
X_raw = data[["Total Fwd Packets"]].values.astype(float)
y_raw = data["Fwd Packets Length Total"].values.astype(float)
 """output:
 Working sample shape (after outlier removal): (5000, 2)
 """

# ----------------------------
# 3) Train/test split (80% train, 20% test)
# ----------------------------
X_train_raw, X_test_raw, y_train_raw, y_test_raw = train_test_split(
    X_raw, y_raw, test_size=0.2, random_state=42
)
# ----------------------------
# 4) Feature scaling (important for SGD -- keeps gradient steps well-behaved)
# ----------------------------
scaler_X = StandardScaler()
X_train = scaler_X.fit_transform(X_train_raw).flatten()
```

```
X_test = scaler_X.transform(X_test_raw).flatten()

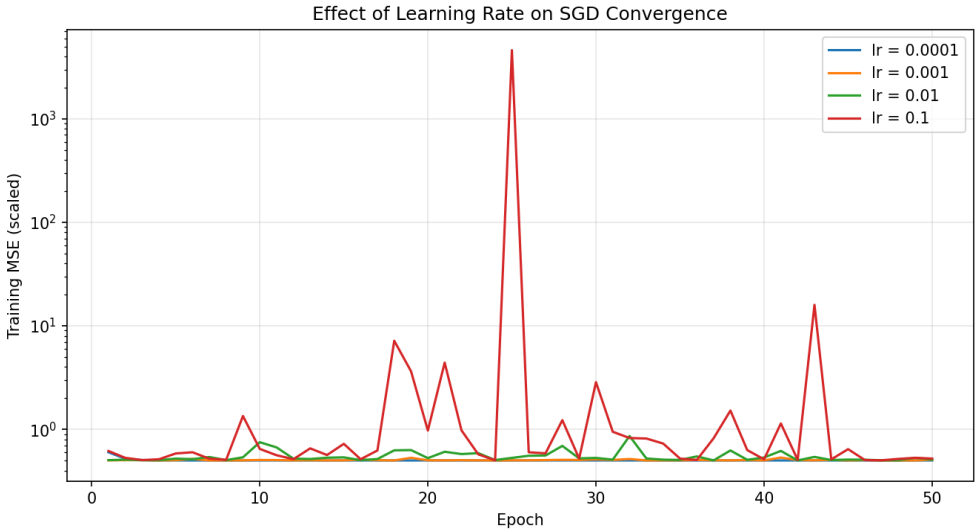
scaler_y = StandardScaler()
y_train = scaler_y.fit_transform(y_train_raw.reshape(-1, 1)).flatten()
y_test = scaler_y.transform(y_test_raw.reshape(-1, 1)).flatten()
# ----------------------------
# 5) SGD training function
# ----------------------------
def train_sgd(X, y, lr, epochs, seed=42):
    """Train y = b0 + b1*x using true stochastic gradient descent
    (one training sample at a time, per the update rule
    theta <- theta - eta * grad(L(theta)))."""
    rng = np.random.default_rng(seed)
    n = len(X)
    b0, b1 = 0.0, 0.0
    loss_history = []
    for epoch in range(epochs):
        indices = rng.permutation(n)  # shuffle each epoch
        for i in indices:
            xi, yi = X[i], y[i]
            y_pred = b0 + b1 * xi
            error = yi - y_pred  # e_i = y_i - y_hat_i
            grad_b0 = -2 * error  # dL/db0
            grad_b1 = -2 * error * xi  # dL/db1
            b0 -= lr * grad_b0
            b1 -= lr * grad_b1
        y_pred_all = b0 + b1 * X
        mse = np.mean((y - y_pred_all) ** 2)
        loss_history.append(mse)
    return b0, b1, loss_history
# ----------------------------
# 6) Train with a chosen learning rate
# ----------------------------
LR = 0.001
EPOCHS = 50
b0, b1, loss_history = train_sgd(X_train, y_train, lr=LR, epochs=EPOCHS)
print(f"\nFinal parameters: b0 = {b0:.4f}, b1 = {b1:.4f}")
print(f"Final training MSE (scaled units): {loss_history[-1]:.4f}")
 """output:
 Final parameters: b0 = 0.0237, b1 = 0.7991
 Final training MSE (scaled units): 0.5092
 """

# ----------------------------
# 7) Plot: convergence (loss vs epoch)
# ----------------------------
plt.figure(figsize=(9, 5))
plt.plot(range(1, EPOCHS + 1), loss_history, marker="o", markersize=3)
plt.xlabel("Epoch")
plt.ylabel("Training MSE (scaled)")
plt.title(f"Convergence of SGD Linear Regression (learning rate = {LR})")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("01_convergence_curve.png", dpi=150)
plt.show()
```




```
# ------------------------------
# 8) Effect of learning rate (experiment)
# ------------------------------
learning_rates = [0.0001, 0.001, 0.01, 0.1]
lr_histories = {}
for lr in learning_rates:
    _, _, hist = train_sgd(X_train, y_train, lr=lr, epochs=EPOCHS)
    lr_histories[lr] = hist
    print(f"lr={lr}: final training MSE = {hist[-1]:.4f}")

plt.figure(figsize=(9, 5))
for lr, hist in lr_histories.items():
    plt.plot(range(1, EPOCHS + 1), hist, label=f"lr = {lr}")
plt.xlabel("Epoch")
plt.ylabel("Training MSE (scaled)")
plt.title("Effect of Learning Rate on SGD Convergence")
plt.yscale("log")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("02_learning_rate_comparison.png", dpi=150)
plt.show()
 """output:
 lr=0.0001: final training MSE = 0.5001
 lr=0.001: final training MSE = 0.5092
 lr=0.01: final training MSE = 0.5104
 lr=0.1: final training MSE = 0.5213
 """
```




```
# ----------------------------
# 9) Predict on test data and evaluate (in original units)
# ----------------------------
y_pred_test_scaled = b0 + b1 * X_test
y_pred_test = scaler_y.inverse_transform(y_pred_test_scaled.reshape(-1, 1)).flatten()
y_test_original = scaler_y.inverse_transform(y_test.reshape(-1, 1)).flatten()

mse = mean_squared_error(y_test_original, y_pred_test)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test_original, y_pred_test)
r2 = r2_score(y_test_original, y_pred_test)

print(f"\nTest set evaluation (original units):")
print(f"MSE  = {mse:.2f}")
print(f"RMSE = {rmse:.2f}")
print(f"MAE  = {mae:.2f}")
print(f"R2 Score = {r2:.4f}")
 """output:
 Test set evaluation (original units):
 MSE  = 253228.51
 RMSE = 503.22
 MAE  = 183.14
 R2 Score = 0.3783
 """

# ----------------------------
# 10) Plot: regression fit on test data
# ----------------------------
order = np.argsort(X_test_raw.flatten())
plt.figure(figsize=(8, 6))
plt.scatter(X_test_raw, y_test_original, s=12, alpha=0.4, label="Actual (test data)")
plt.plot(X_test_raw.flatten()[order], y_pred_test[order], color="red", linewidth=2,
label="SGD fitted line")
plt.xlabel("Total Fwd Packets")
plt.ylabel("Fwd Packets Length Total")
plt.title("Linear Regression via SGD -- Test Set Fit")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("03_regression_fit.png", dpi=150)
plt.show()
```

```
print("\nSaved: 01_convergence_curve.png, 02_learning_rate_comparison.png,
03_regression_fit.png")
 """output:
 Saved: 01_convergence_curve.png, 02_learning_rate_comparison.png,
 03_regression_fit.png
 """
```

